Sentiment Analysis of ChatGPT-related Tweets

Define the problem

This dataset of ChatGPT-related Tweets offers a snapshot of the public conversation around ChatGPT on Twitter. It comprises a CSV file with data on half a million tweets, tracking keywords, hashtags, mentions, and other relevant information. This dataset provides valuable training data for deep learning models aimed at understanding public perception and discourse around conversational AI. Researchers can leverage this data to train models for various tasks, including sentiment analysis to gauge public opinion, trend identification to uncover evolving topics, user engagement prediction to understand interaction drivers, and assessment of AI event impact on public discourse.

The input data from this dataset will be the 500k text of tweets, specifically the cleaned tweet content. We are trying to predict the sentiment of a tweet, which can be positive, neutral, or negative. This is a supervised classification problem, specifically a multi-class single-label classification task. It is multiclass as there are three sentiment categories in the dataset: positive, neutral, and negative. Each tweet can be classified into one of these three classes, so it's a multiclass problem. It is single-label as each tweet is assigned to one sentiment class only. For instance, a tweet cannot be both positive and neutral at the same time. This makes it a single-label classification, as each instance (tweet) has only one label (sentiment).

Choosing a measure of success

For sentiment analysis on tweets, where the dataset may be imbalanced (e.g., more neutral tweets than positive or negative ones), F1-score, precision, and recall are the most appropriate evaluation metrics. Precision measures how many of the tweets predicted as a certain sentiment (e.g., positive) are actually correct, making it important when false positives should be minimized. Recall quantifies how many actual tweets of a given sentiment were correctly identified, which is crucial when missing certain sentiments (false negatives) is costly. F1-score is the harmonic mean of precision and recall, providing a balanced measure that is particularly useful when both false positives and false negatives need to be minimized. Since sentiment classification involves multiple classes (positive, neutral, and negative), a macro-averaged or weighted F1-score ensures fair evaluation across all classes, even if the dataset is imbalanced. This approach avoids misleading results that could arise from using accuracy, and it is preferred over ROC AUC, which is more suited for binary classification.

Accuracy is not used because it can be misleading when dealing with imbalanced datasets. If the dataset contains significantly more neutral tweets than positive or negative ones, a model that simply predicts "neutral" most of the time could achieve high accuracy while failing to correctly classify positive or negative sentiments. This would give a false impression of good performance, even though the model is ineffective at distinguishing between classes. Also, ROC AUC focuses on ranking predictions rather than directly assessing the model's ability to classify sentiments correctly. Since sentiment analysis typically involves positive, neutral, and negative classes, precision, recall, and F1-score provide more practical and interpretable insights into model performance.

Deciding on an evaluation protocol

For this dataset with 500K tweets, hold-out validation is preferred over K-Fold Cross-Validation and iterated K-Fold Cross-Validation due to its computational efficiency. Hold-out validation splits the data into training, validation, and test sets, and trains the model just once, which is much faster than K-Fold, where the model is trained K times. Given the large size of the dataset, K-Fold could be computationally expensive and time-consuming. Additionally, iterated K-Fold would further compound the issue by repeating K-Fold multiple times, making it infeasible for such a large dataset. Therefore, hold-out validation provides a good balance of model evaluation speed and reliability for this scenario, making it the most practical choice for efficient experimentation and testing.

In hold-out validation, the data is split 80% for training, 10% for validation, and 10% for testing. The model is trained on the training set, which contains the majority of the data. After training, the model's performance is evaluated on the validation set, which acts as a proxy for unseen data, allowing for hyperparameter tuning and adjustments. Finally, once the model is fine-tuned using the validation set, its performance is assessed on the test set, which has never been used in the training process, to give an unbiased estimate of its generalization ability.

Data Cleaning

Clean data is crucial because dirty data leads to inaccurate insights and flawed models. Cleaning ensures data is consistent, accurate, and complete, which is essential for reliable analysis, and building high-performing machine learning models.

In [1]:
```
import pandas as pd
import re
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
import nltk

# Download necessary NLTK data
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('wordnet')
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
```

Out[1]: True

In [2]:
```
# Load the dataset
df = pd.read_csv('/Twitter Jan Mar.csv')

# Display basic information about the dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500036 entries, 0 to 500035
Data columns (total 6 columns):
 #   Column         Non-Null Count   Dtype
---  ------         --------------   -----
 0   date           500036 non-null  object
 1   id             500030 non-null  object
 2   content        500030 non-null  object
 3   username       500002 non-null  object
 4   like_count     499974 non-null  float64
 5   retweet_count  499974 non-null  float64
dtypes: float64(2), object(4)
memory usage: 22.9+ MB
```

In [3]:
```
# Display first 5 rows of table
df.head()
```

Out[3]:

	date	id	content	username	like_count	retweet_count
0	2023-03-29 22:58:21+00:00	1641213230730051584	Free AI marketing and automation tools, strate...	RealProfitPros	0.0	0.0
1	2023-03-29 22:58:18+00:00	1641213218520481805	@MecoleHardman4 Chat GPT says it's 15. 😂	AmyLouWho321	0.0	0.0
2	2023-03-29 22:57:53+00:00	1641213115684536323	https://t.co/FjJSprt0te - Chat with any PDF!\n...	yjleon1976	0.0	0.0
3	2023-03-29 22:57:52+00:00	1641213110915571715	AI muses: "In the court of life, we must all f...	ChatGPT_Thinks	0.0	0.0
4	2023-03-29 22:57:26+00:00	1641213003260633088	Most people haven't heard of Chat GPT yet.\nFi...	nikocosmonaut	0.0	0.0

In [4]:

```
df = df.drop("id", axis=1) #id not providing any useful information

# Display first 5 rows of table
df.head()
```

Out[4]:

	date	content	username	like_count	retweet_count
0	2023-03-29 22:58:21+00:00	Free AI marketing and automation tools, strate...	RealProfitPros	0.0	0.0
1	2023-03-29 22:58:18+00:00	@MecoleHardman4 Chat GPT says it's 15. 😂	AmyLouWho321	0.0	0.0
2	2023-03-29 22:57:53+00:00	https://t.co/FjJSprt0te - Chat with any PDF!\n...	yjleon1976	0.0	0.0
3	2023-03-29 22:57:52+00:00	AI muses: "In the court of life, we must all f...	ChatGPT_Thinks	0.0	0.0
4	2023-03-29 22:57:26+00:00	Most people haven't heard of Chat GPT yet.\nFi...	nikocosmonaut	0.0	0.0

In [5]:

```
# Check for any null values
print(df.isnull().sum())
```

```
date              0
content           6
username         34
like_count       62
retweet_count    62
dtype: int64
```

In [6]:

```
# Drop rows with missing values
df = df.dropna()

# Check for any null values again
print(df.isnull().sum())
```

```
date             0
content          0
username         0
like_count       0
retweet_count    0
dtype: int64
```

In [7]:

```
# Remove duplicate data
df = df.drop_duplicates()
```

In [8]:

```
def clean_tweet(tweet):
    # Convert to lowercase
    tweet = tweet.lower()
    # Remove URLs
    tweet = re.sub(r'http\S+|www\S+|https\S+', '', tweet, flags=re.MULTILINE)
    # Remove user @ references and '#'
    tweet = re.sub(r'\@\w+|\#','', tweet)
    # Remove punctuation
    tweet = re.sub(r'[^\w\s]', '', tweet)
    # Tokenize the tweet
    tokens = word_tokenize(tweet)
    # Remove stop words
    tokens = [word for word in tokens if word.lower() not in stopwords.words('english')]
    # Lemmatize words
    lemmatizer = WordNetLemmatizer()
    tokens = [lemmatizer.lemmatize(word) for word in tokens]
    return ' '.join(tokens)

# Apply the cleaning function to the 'content' column
df['cleaned_content'] = df['content'].apply(clean_tweet)
```

In [9]:

```
df.head()
```

Out[9]:

	date	content	username	like_count	retweet_count	cleaned_content
0	2023-03-29 22:58:21+00:00	Free AI marketing and automation tools, strate...	RealProfitPros	0.0	0.0	free ai marketing automation tool strategy col...
1	2023-03-29 22:58:18+00:00	@MecoleHardman4 Chat GPT says it's 15. 😂	AmyLouWho321	0.0	0.0	chat gpt say 15
2	2023-03-29 22:57:53+00:00	https://t.co/FjJSprt0te - Chat with any PDF!\n...	yjleon1976	0.0	0.0	chat pdf check new ai quickly answer question ...
3	2023-03-29 22:57:52+00:00	AI muses: "In the court of life, we must all f...	ChatGPT_Thinks	0.0	0.0	ai mus court life must face judge destiny jury...
4	2023-03-29 22:57:26+00:00	Most people haven't heard of Chat GPT yet.\nFi...	nikocosmonaut	0.0	0.0	people havent heard chat gpt yet first elite f...

In [10]:

```
# Save a copy of the dataframe
cleaned_df = df.copy()
```

Sentiment Analysis

Sentiment analysis uses natural language processing (NLP) to determine the emotional tone behind text and identify whether the tweets are positive, negative, or neutral. This is done using the library vaderSentiment which is a sentiment analysis tool. It is designed to specifically handle social media text, excelling at understanding sentiment with nuanced expressions, including emojis and slang, by assigning a positive, negative, and neutral score.

In [16]:

```
# Install vaderSentiment library
!pip install nltk vaderSentiment
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.11/dist-packages (3.9.1)
Collecting vaderSentiment
  Downloading vaderSentiment-3.3.2-py2.py3-none-any.whl.metadata (572 bytes)
Requirement already satisfied: click in /usr/local/lib/python3.11/dist-packages (from nltk) (8.1.8)
Requirement already satisfied: joblib in /usr/local/lib/python3.11/dist-packages (from nltk) (1.4.2)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.11/dist-packages (from nltk) (2024.11.6)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from nltk) (4.67.1)
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from vaderSentiment) (2.32.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests->vaderSentiment) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests->vaderSentiment) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests->vaderSentiment) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests->vaderSentiment) (2025.1.31)
Downloading vaderSentiment-3.3.2-py2.py3-none-any.whl (125 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 126.0/126.0 kB 2.0 MB/s eta 0:00:00
Installing collected packages: vaderSentiment
Successfully installed vaderSentiment-3.3.2
```

In [17]:
```python
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
```

In [18]:
```python
# Initialize the VADER sentiment analyzer
analyzer = SentimentIntensityAnalyzer()

# Function to classify sentiment based on VADER scores
def get_sentiment_score(tweet):
    sentiment_score = analyzer.polarity_scores(tweet)
    # Return the sentiment category
    if sentiment_score['compound'] >= 0.05:
        return 'positive'
    elif sentiment_score['compound'] <= -0.05:
        return 'negative'
    else:
        return 'neutral'

# Apply the function to the 'cleaned_content' column
cleaned_df['sentiment'] = cleaned_df['cleaned_content'].apply(get_sentiment_score)
```

In [19]:
```python
# Display first 10 rows
cleaned_df[['cleaned_content', 'sentiment']].head(10)
```

Out[19]:

	cleaned_content	sentiment
0	free ai marketing automation tool strategy col...	positive
1	chat gpt say 15	neutral
2	chat pdf check new ai quickly answer question ...	positive
3	ai mus court life must face judge destiny jury...	neutral
4	people havent heard chat gpt yet first elite f...	neutral
5	chat gpt putting together amazing recipe	positive
6	yes also chat gpt make generative art easy nee...	positive
7	people havent heard chat gpt yet first elite f...	neutral
8	robotically shaun usher letter note think chat...	positive
9	metaphor limited perception reality many peopl...	negative

In [20]:
```python
# Get sentiment counts and percentages
sentiment_counts = cleaned_df['sentiment'].value_counts()
sentiment_percentages = cleaned_df['sentiment'].value_counts(normalize=True) * 100

# Convert percentages to string format with % symbol
sentiment_df = pd.DataFrame({
    'Sentiment': sentiment_counts.index,
    'Count': sentiment_counts.values,
    'Percentage': sentiment_percentages.map(lambda x: f"{x:.2f}%")  # Format with 2 decimal places and %
})

# Display the DataFrame
print(sentiment_df)
```

```
          Sentiment   Count Percentage
sentiment
positive   positive  261386     52.28%
neutral     neutral  160992     32.20%
negative   negative   77583     15.52%
```

Data Preprocessing

In [21]:
```python
# Save a copy of the cleaned dataframe as tensor_df
tensor_df = cleaned_df.copy()
tensor_df.head()
```

Out[21]:

	date	content	username	like_count	retweet_count	cleaned_content	sentiment
0	2023-03-29 22:58:21+00:00	Free AI marketing and automation tools, strate...	RealProfitPros	0.0	0.0	free ai marketing automation tool strategy col...	positive
1	2023-03-29 22:58:18+00:00	@MecoleHardman4 Chat GPT says it's 15. 😂	AmyLouWho321	0.0	0.0	chat gpt say 15	neutral
2	2023-03-29 22:57:53+00:00	https://t.co/FjJSprt0te - Chat with any PDF!\n...	yjleon1976	0.0	0.0	chat pdf check new ai quickly answer question ...	positive
3	2023-03-29 22:57:52+00:00	AI muses: "In the court of life, we must all f...	ChatGPT_Thinks	0.0	0.0	ai mus court life must face judge destiny jury...	neutral
4	2023-03-29 22:57:26+00:00	Most people haven't heard of Chat GPT yet.\nFi...	nikocosmonaut	0.0	0.0	people havent heard chat gpt yet first elite f...	neutral

The dataset is preprocessed by mapping textual sentiment labels to numerical values, splitting the dataset into training, validation, and test sets, converting text into numerical features using TF-IDF, and computing class weights to handle imbalance.

The sentiment labels ('positive', 'neutral', 'negative') are mapped to numerical values (2, 1, 0) to make them compatible with machine learning models. The dataset is then split using a two-step approach: first, 80% of the data is used for training while 20% is set aside as a temporary set. This temporary set is further split equally into validation and test sets (10% each) to ensure a balanced distribution of sentiment classes.

Since we're restricted to Dense layers, we need fixed-length numerical input. TF-IDF works well for this. TF-IDF is applied to convert text data into numerical form by selecting the top 5000 most relevant words, ensuring that the transformation is consistently applied to the training, validation, and test sets. As the dataset is imbalanced, class weights are computed to assign higher importance to underrepresented classes, preventing the model from favoring majority classes.

In [22]:
```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.utils.class_weight import compute_class_weight

# Map sentiment labels to numerical values
sentiment_map = {'positive': 2, 'neutral': 1, 'negative': 0}
tensor_df['sentiment'] = tensor_df['sentiment'].map(sentiment_map)

# First split (80% training, 20% temp)
X_train, X_temp, y_train, y_temp = train_test_split(tensor_df['cleaned_content'], tensor_df['sentiment'], test_size=0.2, random_state=42)

# Split temp set (20%) into validation (10%) and test (10%)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42)

# Convert text into numerical features using TF-IDF
vectorizer = TfidfVectorizer(max_features=5000)  # Use top 5000 words
X_train_tfidf = vectorizer.fit_transform(X_train)
X_val_tfidf = vectorizer.transform(X_val)
X_test_tfidf = vectorizer.transform(X_test)

# Compute class weights to handle imbalance
class_weights = compute_class_weight(class_weight='balanced', classes=np.unique(y_train), y=y_train)
class_weight_dict = dict(enumerate(class_weights))
print("Class Weights:", class_weight_dict)

# Print dataset sizes
print(f"Training Set: {len(X_train)} samples")
print(f"Validation Set: {len(X_val)} samples")
print(f"Test Set: {len(X_test)} samples")
```

```
Class Weights: {0: 2.144485550372634, 1: 1.0338937485782824, 2: 0.6383778797136008}
Training Set: 399968 samples
Validation Set: 49996 samples
Test Set: 49997 samples
```

Developing a model that does better than a baseline

To develop a small model that aims to outperform a simple baseline, we need to first establish what the baseline is, then build a simple model, and finally evaluate the performance.

Baseline model

The baseline model used in this context is a DummyClassifier that predicts the most frequent sentiment class in the dataset, which is commonly known as a majority class classifier. This model is implemented as a benchmark to evaluate the performance of more sophisticated models. It serves as a reference point, where the model simply predicts the most common sentiment (e.g., "positive") for all instances in the test set, regardless of the input data. By establishing this baseline, we can gauge whether a more complex model can outperform this simplistic approach, and how well it can capture meaningful patterns in the data.

In [23]:
```
from sklearn.dummy import DummyClassifier
from sklearn.metrics import precision_score, recall_score, f1_score

# Create a dummy classifier
dummy_clf = DummyClassifier(strategy='most_frequent', random_state=42)

# Train the dummy classifier on the training data
dummy_clf.fit(X_train_tfidf, y_train)

# Make predictions on the test data
y_pred_dummy = dummy_clf.predict(X_test_tfidf)

# Evaluate performance using precision, recall, and F1 score
precision = precision_score(y_test, y_pred_dummy, average='weighted', zero_division=1)
recall = recall_score(y_test, y_pred_dummy, average='weighted', zero_division=1)
f1 = f1_score(y_test, y_pred_dummy, average='weighted', zero_division=1)

# Print evaluation metrics
print(f"Dummy Classifier Performance:")
print(f"Precision: {precision}")
print(f"Recall: {recall}")
print(f"F1 Score: {f1}")
```

```
Dummy Classifier Performance:
Precision: 0.7505332120835304
Recall: 0.523091385483129
F1 Score: 0.3593016153523381
```

Small SLP model

This simple single-layer perceptron (SLP) model is implemented with a single dense layer with softmax activation. It is compiled with categorical crossentropy loss, which is suitable for multiclass classification, rmsprop optimizer with a learning_rate=0.001 and evaluated using precision, recall, and F1 score as metrics. It aims to outperform the simpler baseline model by using these metrics, providing a clearer understanding of the model's ability to distinguish between classes.

In [24]:
```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Input  # Layer used to define the shape of the input data for the model
from tensorflow.keras.utils import to_categorical
from sklearn.metrics import precision_score, recall_score, f1_score
from tensorflow.keras.optimizers import RMSprop

# One-hot encode the target labels
y_train_one_hot = to_categorical(y_train, num_classes=3)
y_val_one_hot = to_categorical(y_val, num_classes=3)
y_test_one_hot = to_categorical(y_test, num_classes=3)

# Define the model
```

```
model = Sequential([
    Input(shape=(X_train_tfidf.shape[1],)),  # Specify input shape using Input layer
    Dense(3, activation='softmax')  # Dense layer with softmax activation
])

# Compile the model
model.compile(optimizer=RMSprop(learning_rate=0.001),  # RMSprop optimizer with learning rate
              loss='categorical_crossentropy',  # Categorical crossentropy for multiclass classification
              metrics=['Precision', 'Recall'])  # Precision and Recall as evaluation metrics

# Train the model
history = model.fit(X_train_tfidf, y_train_one_hot, epochs=5, batch_size=32,
                    validation_data=(X_val_tfidf, y_val_one_hot), class_weight=class_weight_dict)

# Evaluate the model on the test set
test_loss, test_precision, test_recall = model.evaluate(X_test_tfidf, y_test_one_hot, verbose=2)

# Make predictions on the test set
y_pred = model.predict(X_test_tfidf)

# Compute F1 Score
f1 = f1_score(y_test_one_hot.argmax(axis=1), y_pred.argmax(axis=1), average='weighted')  # argmax to get class predictions

# Print test results
print(f"Test Precision: {test_precision}")
print(f"Test Recall: {test_recall}")
print(f"Test F1 Score: {f1}")
```

```
Epoch 1/5
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 46s 4ms/step - Precision: 0.8925 - Recall: 0.2608 - loss: 0.8939 - val_Precision: 0.8584 - val_Recall: 0.6662 - val_loss: 0.6099
Epoch 2/5
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 78s 3ms/step - Precision: 0.8572 - Recall: 0.6955 - loss: 0.6147 - val_Precision: 0.8482 - val_Recall: 0.7513 - val_loss: 0.5366
Epoch 3/5
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 85s 4ms/step - Precision: 0.8530 - Recall: 0.7652 - loss: 0.5537 - val_Precision: 0.8488 - val_Recall: 0.7853 - val_loss: 0.5087
Epoch 4/5
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 79s 3ms/step - Precision: 0.8530 - Recall: 0.7942 - loss: 0.5317 - val_Precision: 0.8494 - val_Recall: 0.8012 - val_loss: 0.4976
Epoch 5/5
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 84s 3ms/step - Precision: 0.8546 - Recall: 0.8100 - loss: 0.5199 - val_Precision: 0.8509 - val_Recall: 0.8124 - val_loss: 0.4926
1563/1563 - 4s - 2ms/step - Precision: 0.8478 - Recall: 0.8114 - loss: 0.4842
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step
Test Precision: 0.8477870225906372
Test Recall: 0.8114486932754517
Test F1 Score: 0.8362037487574917
```

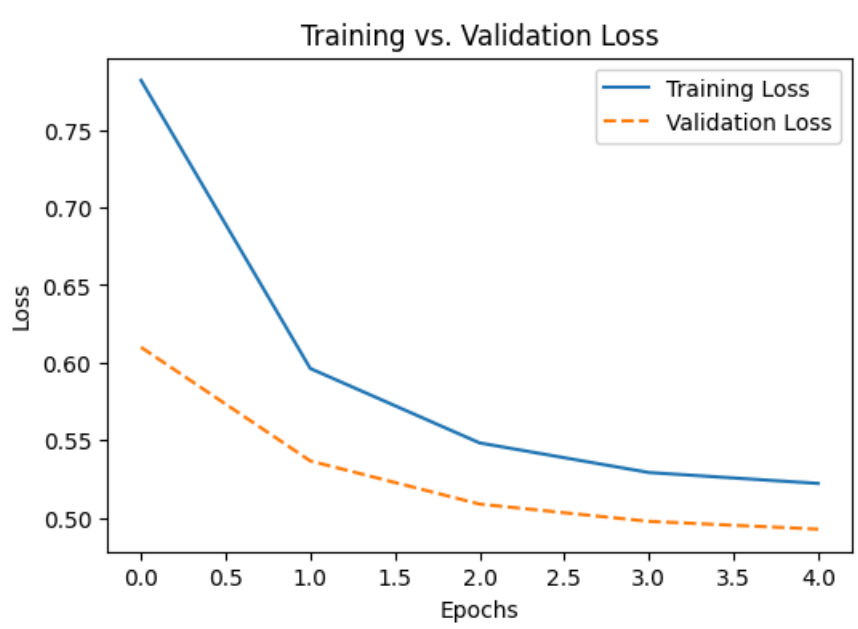
Compared to the Dummy Classifier, the trained model shows a significant improvement in performance. The Dummy Classifier had a precision of 0.75, recall of 0.52, and an F1 score of 0.36, indicating that it was simply predicting the most frequent class without learning meaningful patterns from the data. In contrast, the trained model achieved a precision of 0.85, recall of 0.81, and an F1 score of 0.84 on the test set, reflecting its ability to effectively differentiate between classes and make informed predictions. The trained model significantly outperforms the dummy baseline across all metrics, showcasing that it has successfully learned from the data and generalized well to the test set.

Function defined for plotting graphs

The plot_loss function is a helper function designed to visualize the training and validation loss of a machine learning model over epochs, helping to assess model performance and detect issues like overfitting. By isolating this task into a separate function, it provides a simple and reusable tool for evaluating model behavior, ensuring that the code remains modular and easy to maintain.

In [25]:
```
import matplotlib.pyplot as plt

# Function to plot training & validation loss
def plot_loss(history):
    plt.figure(figsize=(6, 4))
    plt.plot(history.history['loss'], label='Training Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss', linestyle='dashed')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.title('Training vs. Validation Loss')
    plt.legend()
    plt.show()
```

In [26]:
```
# Plotting the training and validation loss of small SLP model
plot_loss(history)
```



Both training and validation loss show a steep decline in the early epochs. This indicates that the model is quickly learning the underlying patterns in the data. The rate of decrease in loss slows down as the number of epochs increases. The validation loss closely follows the training loss throughout the training process. This suggests that the model is generalizing well to unseen data and is not overfitting to the training data. Overfitting would be indicated by a significant divergence between the training and validation loss, where the training loss continues to decrease while the validation loss plateaus or starts to increase.

Scaling up: developing a model that overfits

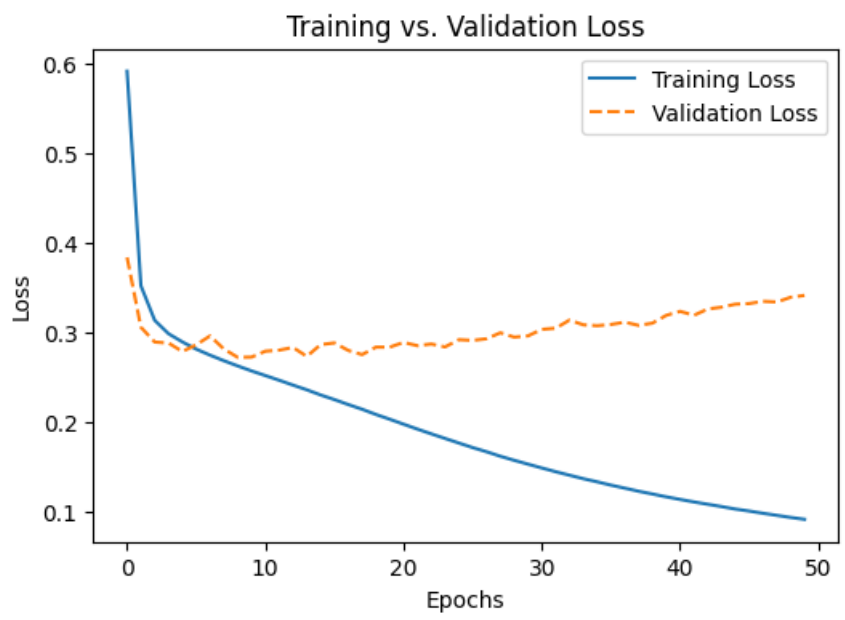
Building an overfit model is an essential step in understanding a model's limitations. By intentionally training a model, often with one or two hidden layers, we can observe how it memorizes the training data instead of generalizing to unseen data. Monitoring both the training and validation loss helps us identify the point at which the model begins overfitting. This approach helps us evaluate the level of complexity required to capture patterns in the data and decide on the appropriate regularization techniques to mitigate overfitting.

In [ ]:
```
# Define a more complex model that is likely to overfit
overfit_model = Sequential([
    Input(shape=(X_train_tfidf.shape[1],)),
    Dense(128, activation='relu'),
    Dense(3, activation='softmax')
])

# Compile the model with RMSprop optimizer
overfit_model.compile(optimizer=RMSprop(learning_rate=0.001),
              loss='categorical_crossentropy',
              metrics=['Precision', 'Recall'])

# Train the model for a large number of epochs
overfit_history = overfit_model.fit(X_train_tfidf, y_train_one_hot, epochs=50, batch_size=512,
                                    validation_data=(X_val_tfidf, y_val_one_hot), class_weight=class_weight_dict)

# Evaluate the model on the test set
test_loss, test_precision, test_recall = overfit_model.evaluate(X_test_tfidf, y_test_one_hot, verbose=2)

# Make predictions on the test set
y_pred = overfit_model.predict(X_test_tfidf)

# Compute F1 Score
f1 = f1_score(y_test_one_hot.argmax(axis=1), y_pred.argmax(axis=1), average='weighted')

# Print test results
print(f"Test Precision: {test_precision}")
print(f"Test Recall: {test_recall}")
print(f"Test F1 Score: {f1}")
```

```
Epoch 1/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 23s 27ms/step - Precision: 0.8696 - Recall: 0.3868 - loss: 0.7883 - val_Precision: 0.9078 - val_Recall: 0.8727 - val_loss: 0.3836
Epoch 2/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 40s 25ms/step - Precision: 0.9163 - Recall: 0.8921 - loss: 0.3615 - val_Precision: 0.9219 - val_Recall: 0.9068 - val_loss: 0.3057
Epoch 3/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 24s 31ms/step - Precision: 0.9234 - Recall: 0.9105 - loss: 0.3156 - val_Precision: 0.9227 - val_Recall: 0.9108 - val_loss: 0.2892
Epoch 4/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 37s 26ms/step - Precision: 0.9261 - Recall: 0.9146 - loss: 0.2950 - val_Precision: 0.9226 - val_Recall: 0.9097 - val_loss: 0.2881
Epoch 5/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 23s 29ms/step - Precision: 0.9289 - Recall: 0.9172 - loss: 0.2884 - val_Precision: 0.9268 - val_Recall: 0.9141 - val_loss: 0.2786
Epoch 6/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 38s 25ms/step - Precision: 0.9321 - Recall: 0.9210 - loss: 0.2763 - val_Precision: 0.9261 - val_Recall: 0.9121 - val_loss: 0.2861
Epoch 7/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 21s 26ms/step - Precision: 0.9354 - Recall: 0.9242 - loss: 0.2698 - val_Precision: 0.9215 - val_Recall: 0.9097 - val_loss: 0.2960
Epoch 8/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 40s 25ms/step - Precision: 0.9370 - Recall: 0.9259 - loss: 0.2643 - val_Precision: 0.9277 - val_Recall: 0.9155 - val_loss: 0.2814
Epoch 9/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 21s 27ms/step - Precision: 0.9399 - Recall: 0.9285 - loss: 0.2576 - val_Precision: 0.9306 - val_Recall: 0.9190 - val_loss: 0.2722
Epoch 10/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 19s 25ms/step - Precision: 0.9402 - Recall: 0.9294 - loss: 0.2541 - val_Precision: 0.9319 - val_Recall: 0.9188 - val_loss: 0.2724
Epoch 11/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 22s 27ms/step - Precision: 0.9432 - Recall: 0.9327 - loss: 0.2468 - val_Precision: 0.9293 - val_Recall: 0.9162 - val_loss: 0.2790
Epoch 12/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 40s 25ms/step - Precision: 0.9431 - Recall: 0.9324 - loss: 0.2433 - val_Precision: 0.9282 - val_Recall: 0.9168 - val_loss: 0.2801
Epoch 13/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 21s 27ms/step - Precision: 0.9445 - Recall: 0.9340 - loss: 0.2372 - val_Precision: 0.9276 - val_Recall: 0.9158 - val_loss: 0.2832
Epoch 14/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 40s 25ms/step - Precision: 0.9459 - Recall: 0.9358 - loss: 0.2335 - val_Precision: 0.9326 - val_Recall: 0.9188 - val_loss: 0.2732
Epoch 15/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 21s 26ms/step - Precision: 0.9478 - Recall: 0.9378 - loss: 0.2260 - val_Precision: 0.9268 - val_Recall: 0.9159 - val_loss: 0.2863
Epoch 16/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 20s 25ms/step - Precision: 0.9489 - Recall: 0.9390 - loss: 0.2211 - val_Precision: 0.9279 - val_Recall: 0.9146 - val_loss: 0.2882
Epoch 17/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 20s 25ms/step - Precision: 0.9505 - Recall: 0.9409 - loss: 0.2164 - val_Precision: 0.9299 - val_Recall: 0.9184 - val_loss: 0.2798
Epoch 18/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 20s 25ms/step - Precision: 0.9527 - Recall: 0.9432 - loss: 0.2098 - val_Precision: 0.9309 - val_Recall: 0.9201 - val_loss: 0.2752
Epoch 19/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 21s 25ms/step - Precision: 0.9530 - Recall: 0.9441 - loss: 0.2056 - val_Precision: 0.9289 - val_Recall: 0.9177 - val_loss: 0.2835
Epoch 20/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 21s 25ms/step - Precision: 0.9555 - Recall: 0.9469 - loss: 0.1963 - val_Precision: 0.9295 - val_Recall: 0.9188 - val_loss: 0.2835
Epoch 21/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 20s 25ms/step - Precision: 0.9566 - Recall: 0.9485 - loss: 0.1906 - val_Precision: 0.9275 - val_Recall: 0.9155 - val_loss: 0.2888
Epoch 22/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 19s 25ms/step - Precision: 0.9576 - Recall: 0.9495 - loss: 0.1885 - val_Precision: 0.9297 - val_Recall: 0.9186 - val_loss: 0.2851
Epoch 23/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 21s 25ms/step - Precision: 0.9589 - Recall: 0.9513 - loss: 0.1835 - val_Precision: 0.9278 - val_Recall: 0.9175 - val_loss: 0.2870
Epoch 24/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 21s 26ms/step - Precision: 0.9598 - Recall: 0.9522 - loss: 0.1783 - val_Precision: 0.9299 - val_Recall: 0.9199 - val_loss: 0.2837
Epoch 25/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 21s 26ms/step - Precision: 0.9622 - Recall: 0.9553 - loss: 0.1724 - val_Precision: 0.9274 - val_Recall: 0.9169 - val_loss: 0.2918
Epoch 26/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 40s 25ms/step - Precision: 0.9633 - Recall: 0.9566 - loss: 0.1649 - val_Precision: 0.9269 - val_Recall: 0.9168 - val_loss: 0.2910
Epoch 27/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 21s 25ms/step - Precision: 0.9646 - Recall: 0.9583 - loss: 0.1613 - val_Precision: 0.9271 - val_Recall: 0.9164 - val_loss: 0.2928
Epoch 28/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 20s 25ms/step - Precision: 0.9658 - Recall: 0.9597 - loss: 0.1593 - val_Precision: 0.9250 - val_Recall: 0.9157 - val_loss: 0.2995
Epoch 29/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 20s 25ms/step - Precision: 0.9668 - Recall: 0.9610 - loss: 0.1555 - val_Precision: 0.9267 - val_Recall: 0.9174 - val_loss: 0.2946
Epoch 30/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 19s 24ms/step - Precision: 0.9684 - Recall: 0.9629 - loss: 0.1491 - val_Precision: 0.9269 - val_Recall: 0.9177 - val_loss: 0.2961
Epoch 31/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 20s 25ms/step - Precision: 0.9700 - Recall: 0.9647 - loss: 0.1423 - val_Precision: 0.9248 - val_Recall: 0.9160 - val_loss: 0.3033
Epoch 32/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 20s 25ms/step - Precision: 0.9707 - Recall: 0.9658 - loss: 0.1384 - val_Precision: 0.9257 - val_Recall: 0.9160 - val_loss: 0.3046
Epoch 33/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 19s 24ms/step - Precision: 0.9713 - Recall: 0.9664 - loss: 0.1369 - val_Precision: 0.9220 - val_Recall: 0.9139 - val_loss: 0.3136
Epoch 34/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 21s 25ms/step - Precision: 0.9721 - Recall: 0.9676 - loss: 0.1331 - val_Precision: 0.9245 - val_Recall: 0.9160 - val_loss: 0.3084
Epoch 35/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 20s 25ms/step - Precision: 0.9734 - Recall: 0.9692 - loss: 0.1275 - val_Precision: 0.9246 - val_Recall: 0.9162 - val_loss: 0.3074
Epoch 36/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 21s 25ms/step - Precision: 0.9740 - Recall: 0.9703 - loss: 0.1250 - val_Precision: 0.9244 - val_Recall: 0.9162 - val_loss: 0.3087
Epoch 37/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 19s 24ms/step - Precision: 0.9748 - Recall: 0.9709 - loss: 0.1225 - val_Precision: 0.9237 - val_Recall: 0.9158 - val_loss: 0.3113
Epoch 38/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 21s 25ms/step - Precision: 0.9758 - Recall: 0.9720 - loss: 0.1190 - val_Precision: 0.9256 - val_Recall: 0.9179 - val_loss: 0.3077
Epoch 39/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 20s 25ms/step - Precision: 0.9764 - Recall: 0.9731 - loss: 0.1156 - val_Precision: 0.9245 - val_Recall: 0.9167 - val_loss: 0.3102
Epoch 40/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 21s 25ms/step - Precision: 0.9771 - Recall: 0.9740 - loss: 0.1129 - val_Precision: 0.9223 - val_Recall: 0.9147 - val_loss: 0.3190
Epoch 41/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 20s 25ms/step - Precision: 0.9781 - Recall: 0.9749 - loss: 0.1100 - val_Precision: 0.9217 - val_Recall: 0.9141 - val_loss: 0.3234
Epoch 42/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 20s 24ms/step - Precision: 0.9786 - Recall: 0.9758 - loss: 0.1058 - val_Precision: 0.9228 - val_Recall: 0.9158 - val_loss: 0.3190
Epoch 43/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 20s 25ms/step - Precision: 0.9799 - Recall: 0.9772 - loss: 0.1021 - val_Precision: 0.9214 - val_Recall: 0.9139 - val_loss: 0.3260
Epoch 44/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 20s 25ms/step - Precision: 0.9799 - Recall: 0.9773 - loss: 0.1019 - val_Precision: 0.9205 - val_Recall: 0.9138 - val_loss: 0.3280
Epoch 45/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 21s 25ms/step - Precision: 0.9803 - Recall: 0.9779 - loss: 0.0994 - val_Precision: 0.9196 - val_Recall: 0.9127 - val_loss: 0.3314
Epoch 46/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 20s 25ms/step - Precision: 0.9810 - Recall: 0.9786 - loss: 0.0957 - val_Precision: 0.9198 - val_Recall: 0.9130 - val_loss: 0.3321
Epoch 47/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 21s 25ms/step - Precision: 0.9817 - Recall: 0.9794 - loss: 0.0953 - val_Precision: 0.9197 - val_Recall: 0.9133 - val_loss: 0.3346
Epoch 48/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 21s 25ms/step - Precision: 0.9822 - Recall: 0.9799 - loss: 0.0912 - val_Precision: 0.9206 - val_Recall: 0.9141 - val_loss: 0.3338
Epoch 49/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 32s 39ms/step - Precision: 0.9827 - Recall: 0.9806 - loss: 0.0892 - val_Precision: 0.9186 - val_Recall: 0.9121 - val_loss: 0.3391
Epoch 50/50
782/782 ━━━━━━━━━━━━━━━━━━━━ 29s 24ms/step - Precision: 0.9829 - Recall: 0.9810 - loss: 0.0893 - val_Precision: 0.9182 - val_Recall: 0.9117 - val_loss: 0.3412
1563/1563 - 4s - 2ms/step - Precision: 0.9184 - Recall: 0.9114 - loss: 0.3416
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step
Test Precision: 0.9184319376945496
Test Recall: 0.9114146828651428
Test F1 Score: 0.915016595740849
```

In [ ]:

```
# Plotting the training and validation loss of overfit model
plot_loss(overfit_history)
```



The model achieved a precision of 0.92, a recall of 0.91, and an F1 score of 0.92, with 50 epochs and a batch size of 512. The model begins to overfit after 10 epochs. During the first few epochs, both training and validation performance improve consistently, with the training and validation losses decreasing steadily. However, around epoch 10 - 15, while the training loss continues decreasing, the validation loss starts to increase, indicating that the model is no longer generalizing well to the validation data. This gap between the training and validation losses becomes more pronounced after epoch 15, suggesting that the model has started to overfit to the training data. To prevent further overfitting, it would be best to either stop at epoch 10 and apply regularization techniques.

Regularizing model and tuning hyperparameters

Regularization

It is identified that the model overfits at epoch 10. To address the issue of overfitting, regularization techniques like dropout and L2 regularization can be very effective. Dropout works by randomly deactivating a fraction of neurons during each training step, forcing the model to learn more robust features and preventing it from becoming overly reliant on specific neurons. This reduces the likelihood of the model memorizing the training data, thus improving its ability to generalize to unseen data. On the other hand, L2 regularization (also known as weight decay) adds a penalty term to the loss function that discourages large weights by penalizing the sum of squared weights. This helps prevent the model from fitting noise or outliers in the training data, making it more stable and less prone to overfitting. By combining these two regularization techniques, the model can achieve better generalization, as it learns to make more reliable predictions while avoiding the risk of overfitting to the training data.

Hyperparameter tuning

To find the optimal hyperparameters, Optuna is used. Optuna is a tool that helps to optimize the performance of the model by searching for the ideal combination of hyperparameters.

In [28]:
```
# Install optuna library
!pip install optuna
```

```
Collecting optuna
  Downloading optuna-4.2.1-py3-none-any.whl.metadata (17 kB)
Collecting alembic>=1.5.0 (from optuna)
  Downloading alembic-1.15.1-py3-none-any.whl.metadata (7.2 kB)
Collecting colorlog (from optuna)
  Downloading colorlog-6.9.0-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from optuna) (1.26.4)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-packages (from optuna) (24.2)
Requirement already satisfied: sqlalchemy>=1.4.2 in /usr/local/lib/python3.11/dist-packages (from optuna) (2.0.38)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from optuna) (4.67.1)
Requirement already satisfied: PyYAML in /usr/local/lib/python3.11/dist-packages (from optuna) (6.0.2)
Collecting Mako (from alembic>=1.5.0->optuna)
  Downloading Mako-1.3.9-py3-none-any.whl.metadata (2.9 kB)
Requirement already satisfied: typing-extensions>=4.12 in /usr/local/lib/python3.11/dist-packages (from alembic>=1.5.0->optuna) (4.12.2)
Requirement already satisfied: greenlet!=0.4.17 in /usr/local/lib/python3.11/dist-packages (from sqlalchemy>=1.4.2->optuna) (3.1.1)
Requirement already satisfied: MarkupSafe>=0.9.2 in /usr/local/lib/python3.11/dist-packages (from Mako->alembic>=1.5.0->optuna) (3.0.2)
Downloading optuna-4.2.1-py3-none-any.whl (383 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 383.6/383.6 kB 7.4 MB/s eta 0:00:00
Downloading alembic-1.15.1-py3-none-any.whl (231 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 231.8/231.8 kB 18.7 MB/s eta 0:00:00
Downloading colorlog-6.9.0-py3-none-any.whl (11 kB)
Downloading Mako-1.3.9-py3-none-any.whl (78 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 78.5/78.5 kB 8.4 MB/s eta 0:00:00
Installing collected packages: Mako, colorlog, alembic, optuna
Successfully installed Mako-1.3.9 alembic-1.15.1 colorlog-6.9.0 optuna-4.2.1
```

Conduct a hyperparameter search to identify the best values for dropout, L2 regularization, learning rate and batch size.

In [39]:
```
from tensorflow.keras.layers import Dropout
from tensorflow.keras.regularizers import l2
import optuna

# Define Bayesian Optimization function
def objective(trial):
    # Suggest dropout rate (between 0.1 and 0.5)
    dropout = trial.suggest_float("dropout", 0.1, 0.5)

    # Suggest L2 regularization strength (between 1e-6 and 1e-2)
    l2_strength = trial.suggest_float("l2", 1e-6, 1e-2, log=True)

    # Suggest learning rate
    learning_rate = trial.suggest_float('learning_rate', 1e-5, 1e-2, log=True)

    # Suggest batch size
    batch_size = trial.suggest_categorical('batch_size', [16, 32, 64, 128])

    # Build the model
    regularized_model = Sequential([
        Dense(128, activation="relu", kernel_regularizer=l2(l2_strength)),
        Dropout(dropout),
        Dense(64, activation="relu", kernel_regularizer=l2(l2_strength)),
        Dropout(dropout),
        Dense(y_train_one_hot.shape[1], activation="softmax")  # Output layer
    ])
```

```
    # Compile the model with RMSprop optimizer
    regularized_model.compile(optimizer=RMSprop(learning_rate=0.001),
                              loss="categorical_crossentropy",
                              metrics=["Precision", "Recall"])

    # Train the model
    regularized_history = regularized_model.fit(X_train_tfidf, y_train_one_hot,
                                                validation_data=(X_val_tfidf, y_val_one_hot),
                                                epochs=10,
                                                batch_size=64,
                                                verbose=0)  # Silent training for optimization

    # Return validation loss as the metric to minimize
    return regularized_history.history["val_loss"][-1]

# Run Bayesian Optimization
study = optuna.create_study(direction="minimize", study_name="Hyperparameters optimization")  # Minimize validation loss
study.optimize(objective, n_trials=10)

# Best hyperparameters
print("Best Hyperparameters:", study.best_params)
```

```
[I 2025-03-10 03:52:27,970] A new study created in memory with name: Hyperparameters optimization
[I 2025-03-10 04:05:18,185] Trial 0 finished with value: 0.3083796203136444 and parameters: {'dropout': 0.48753892679696, 'l2': 7.327459773992629e-06, 'learning_ra
te': 0.00012283747275192443, 'batch_size': 32}. Best is trial 0 with value: 0.3083796203136444.
[I 2025-03-10 04:18:40,251] Trial 1 finished with value: 0.49142158031463623 and parameters: {'dropout': 0.24205362980034262, 'l2': 0.0015348138852350173, 'learnin
g_rate': 0.007908599843452114, 'batch_size': 128}. Best is trial 0 with value: 0.3083796203136444.
[I 2025-03-10 04:31:24,110] Trial 2 finished with value: 0.5133467316627502 and parameters: {'dropout': 0.2539639357295416, 'l2': 0.002213302842479591, 'learning_r
ate': 0.0007940670587928886, 'batch_size': 64}. Best is trial 0 with value: 0.3083796203136444.
[I 2025-03-10 04:44:26,310] Trial 3 finished with value: 0.2837753891944885 and parameters: {'dropout': 0.4412888692865231, 'l2': 1.5180064910125624e-06, 'learning
_rate': 0.005719948770296867, 'batch_size': 128}. Best is trial 3 with value: 0.2837753891944885.
[I 2025-03-10 04:57:36,180] Trial 4 finished with value: 0.33643215894699097 and parameters: {'dropout': 0.2927694115711792, 'l2': 8.893448677154588e-05, 'learning
_rate': 0.00010630192431367113, 'batch_size': 128}. Best is trial 3 with value: 0.2837753891944885.
[I 2025-03-10 05:11:09,063] Trial 5 finished with value: 0.529485285282135 and parameters: {'dropout': 0.2495318538664929, 'l2': 0.003926963864933343, 'learning_ra
te': 0.0015877753308518926, 'batch_size': 128}. Best is trial 3 with value: 0.2837753891944885.
[I 2025-03-10 05:26:49,645] Trial 6 finished with value: 0.2786848247051239 and parameters: {'dropout': 0.36630647973293085, 'l2': 2.7982644952859838e-06, 'learnin
g_rate': 0.00022883303500361286, 'batch_size': 32}. Best is trial 6 with value: 0.2786848247051239.
[I 2025-03-10 05:39:59,347] Trial 7 finished with value: 0.2971171438694 and parameters: {'dropout': 0.2561637740417564, 'l2': 2.5668698650344916e-05, 'learning_ra
te': 8.181982042131233e-05, 'batch_size': 128}. Best is trial 6 with value: 0.2786848247051239.
[I 2025-03-10 05:54:19,129] Trial 8 finished with value: 0.40139898657798767 and parameters: {'dropout': 0.30732979503319086, 'l2': 0.00020107950614336513, 'learni
ng_rate': 0.00042778319610281015, 'batch_size': 16}. Best is trial 6 with value: 0.2786848247051239.
[I 2025-03-10 06:09:00,483] Trial 9 finished with value: 0.4306493103504181 and parameters: {'dropout': 0.3403730808053612, 'l2': 0.0004973418848843128, 'learning_
rate': 0.0019289745328029163, 'batch_size': 32}. Best is trial 6 with value: 0.2786848247051239.
Best Hyperparameters: {'dropout': 0.36630647973293085, 'l2': 2.7982644952859838e-06, 'learning_rate': 0.00022883303500361286, 'batch_size': 32}
```

Based on the results of the hyperparameter optimization study, Trial 6 achieved the lowest validation loss of 0.2786848247051239, indicating the best generalization among all tested configurations. The optimal dropout rate of 0.366 indicates a moderate level of regularization, suggesting that while some degree of overfitting mitigation is necessary, excessive dropout is not required. This balance likely allows the model to retain sufficient complexity to learn intricate patterns without becoming overly specialized to the training data. The extremely low L2 regularization value of 2.79e-06 signifies that the model is inherently resistant to overfitting, or that other forms of regularization (like dropout) are effectively handling it. This minimal L2 penalty allows the model to freely adjust its weights without significant constraints. The optimal learning rate of 0.000228 points to a deliberate and controlled learning process. This rate allows the model to make gradual adjustments to its parameters, avoiding overshooting or getting stuck in local minima, which is especially important when the loss landscape is complex. Finally, the batch size of 32 suggests that the model benefits from processing smaller chunks of data at a time. This could facilitate more frequent parameter updates, leading to faster convergence and potentially better generalization. In essence, these best parameters collectively indicate the importance of fine-tuning for optimal performance.

Testing the regularized model

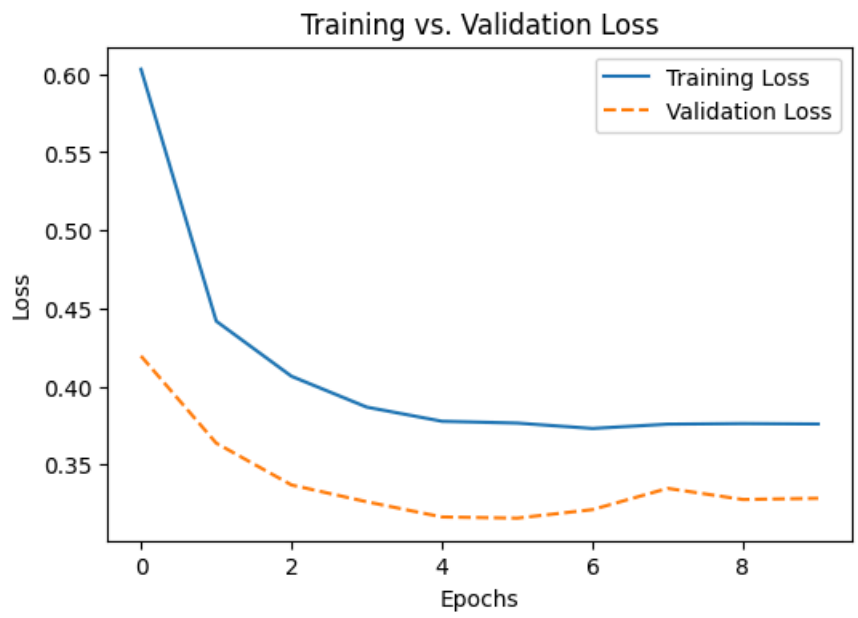
Using the best hyperparameters, we retrained the regularized model.

In [40]:

```
# Retrieve the best hyperparameters
best_dropout = study.best_params["dropout"]
best_l2 = study.best_params["l2"]
best_learning_rate = study.best_params["learning_rate"]
best_batch_size = study.best_params["batch_size"]

# Define the model with the best hyperparameters
regularized_model = Sequential([
    Dense(128, activation="relu", kernel_regularizer=l2(best_l2)),
    Dropout(best_dropout),
    Dense(64, activation="relu", kernel_regularizer=l2(best_l2)),
    Dropout(best_dropout),
    Dense(y_train_one_hot.shape[1], activation="softmax")  # Output layer
])

# Compile the model with the best learning rate
regularized_model.compile(optimizer=RMSprop(learning_rate=best_learning_rate),
                          loss="categorical_crossentropy",
                          metrics=["Precision", "Recall"])

# Train the model with the best batch size
regularized_history = regularized_model.fit(X_train_tfidf, y_train_one_hot, epochs=10, batch_size=best_batch_size,
                                            validation_data=(X_val_tfidf, y_val_one_hot), class_weight=class_weight_dict
)

# Make predictions on the test set
y_pred = regularized_model.predict(X_test_tfidf)

# Compute F1 Score
regularized_f1 = f1_score(y_test_one_hot.argmax(axis=1), y_pred.argmax(axis=1), average='weighted')

# Evaluate on test data
regularized_test_loss, regularized_test_precision, regularized_test_recall = regularized_model.evaluate(X_test_tfidf, y_test_one_hot)

# Print test results
print(f"Test Precision: {regularized_test_precision}")
print(f"Test Recall: {regularized_test_recall}")
print(f"Test F1 Score: {regularized_f1}")
```

```
Epoch 1/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 137s 11ms/step - Precision: 0.8177 - Recall: 0.5167 - loss: 0.7533 - val_Precision: 0.8854 - val_Recall: 0.8475 - val_loss: 0.4195
Epoch 2/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 142s 11ms/step - Precision: 0.8912 - Recall: 0.8527 - loss: 0.4529 - val_Precision: 0.9067 - val_Recall: 0.8801 - val_loss: 0.3637
Epoch 3/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 141s 11ms/step - Precision: 0.9076 - Recall: 0.8800 - loss: 0.4120 - val_Precision: 0.9158 - val_Recall: 0.8976 - val_loss: 0.3369
Epoch 4/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 141s 11ms/step - Precision: 0.9183 - Recall: 0.8966 - loss: 0.3857 - val_Precision: 0.9209 - val_Recall: 0.9040 - val_loss: 0.3261
Epoch 5/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 135s 11ms/step - Precision: 0.9226 - Recall: 0.9015 - loss: 0.3767 - val_Precision: 0.9267 - val_Recall: 0.9083 - val_loss: 0.3164
Epoch 6/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 132s 11ms/step - Precision: 0.9249 - Recall: 0.9027 - loss: 0.3759 - val_Precision: 0.9284 - val_Recall: 0.9100 - val_loss: 0.3157
Epoch 7/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 143s 11ms/step - Precision: 0.9274 - Recall: 0.9063 - loss: 0.3723 - val_Precision: 0.9299 - val_Recall: 0.9103 - val_loss: 0.3211
Epoch 8/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 140s 11ms/step - Precision: 0.9298 - Recall: 0.9079 - loss: 0.3746 - val_Precision: 0.9277 - val_Recall: 0.9063 - val_loss: 0.3347
Epoch 9/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 130s 10ms/step - Precision: 0.9304 - Recall: 0.9074 - loss: 0.3750 - val_Precision: 0.9303 - val_Recall: 0.9105 - val_loss: 0.3276
Epoch 10/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 130s 10ms/step - Precision: 0.9309 - Recall: 0.9087 - loss: 0.3755 - val_Precision: 0.9306 - val_Recall: 0.9118 - val_loss: 0.3284
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - Precision: 0.9313 - Recall: 0.9120 - loss: 0.3220
Test Precision: 0.9308803081512451
Test Recall: 0.9118146896362305
Test F1 Score: 0.9200080230674501
```

In [41]:
```
# Plotting the training and validation loss of regularized model
plot_loss(regularized_history)
```



The model achieved a precision of 0.93, a recall of 0.91, and an F1 score of 0.92. The graph shows a decreasing trend in both training and validation loss over the epochs, indicating that the regularized model is learning effectively. The training loss drops steeply at the start and then stabilizes, while the validation loss also decreases but at a slower rate. The small gap between the two suggests that the model generalizes well to unseen data, with no significant signs of overfitting. This indicates that the regularization techniques and hyperparameter tuning effectively mitigated overfitting.

Different architectures

Using the best hyperparameters for batch size, learning rate, L2 regularization and dropout, we will experiment with different architectures and try different numbers of layers, neuron sizes, and activation functions. Then, metrics (precision, recall, f1 score), training loss and validation loss will be evaluated to see which one generalizes best.

In [42]:
```
# Function to build deeper and wider model
def build_model(architecture, input_shape, output_units, dropout, l2_reg, learning_rate):
    model = Sequential()

    if architecture == "deeper":  # DEEPER MODEL (4 Dense Layers)
        model.add(Dense(256, activation="relu", kernel_regularizer=l2(l2_reg), input_shape=input_shape))
        model.add(Dropout(dropout))
        model.add(Dense(128, activation="relu", kernel_regularizer=l2(l2_reg)))
        model.add(Dropout(dropout))
        model.add(Dense(64, activation="relu", kernel_regularizer=l2(l2_reg)))
        model.add(Dropout(dropout))
        model.add(Dense(32, activation="relu", kernel_regularizer=l2(l2_reg)))
        model.add(Dropout(dropout))

    elif architecture == "wider":  # WIDER MODEL (More neurons per layer)
        model.add(Dense(256, activation="relu", kernel_regularizer=l2(l2_reg), input_shape=input_shape))
        model.add(Dropout(dropout))
        model.add(Dense(128, activation="relu", kernel_regularizer=l2(l2_reg)))
        model.add(Dropout(dropout))

    # Add last softmax layer
    model.add(Dense(output_units, activation="softmax"))

    model.compile(optimizer=RMSprop(learning_rate=learning_rate),
                  loss="categorical_crossentropy",
                  metrics=["Precision", "Recall"])

    return model
```

Now, we train the deeper, and wider models with the best hyperparameters found.

In [43]:
```
architectures = ["deeper", "wider"]

results = {
    "regularized": {  # Store regularized model's results (already trained)
        "test_loss": regularized_test_loss,
        "test_precision": regularized_test_precision,
        "test_recall": regularized_test_recall,
        "f1_score": regularized_f1,
        "history": regularized_history.history
    }
}
```

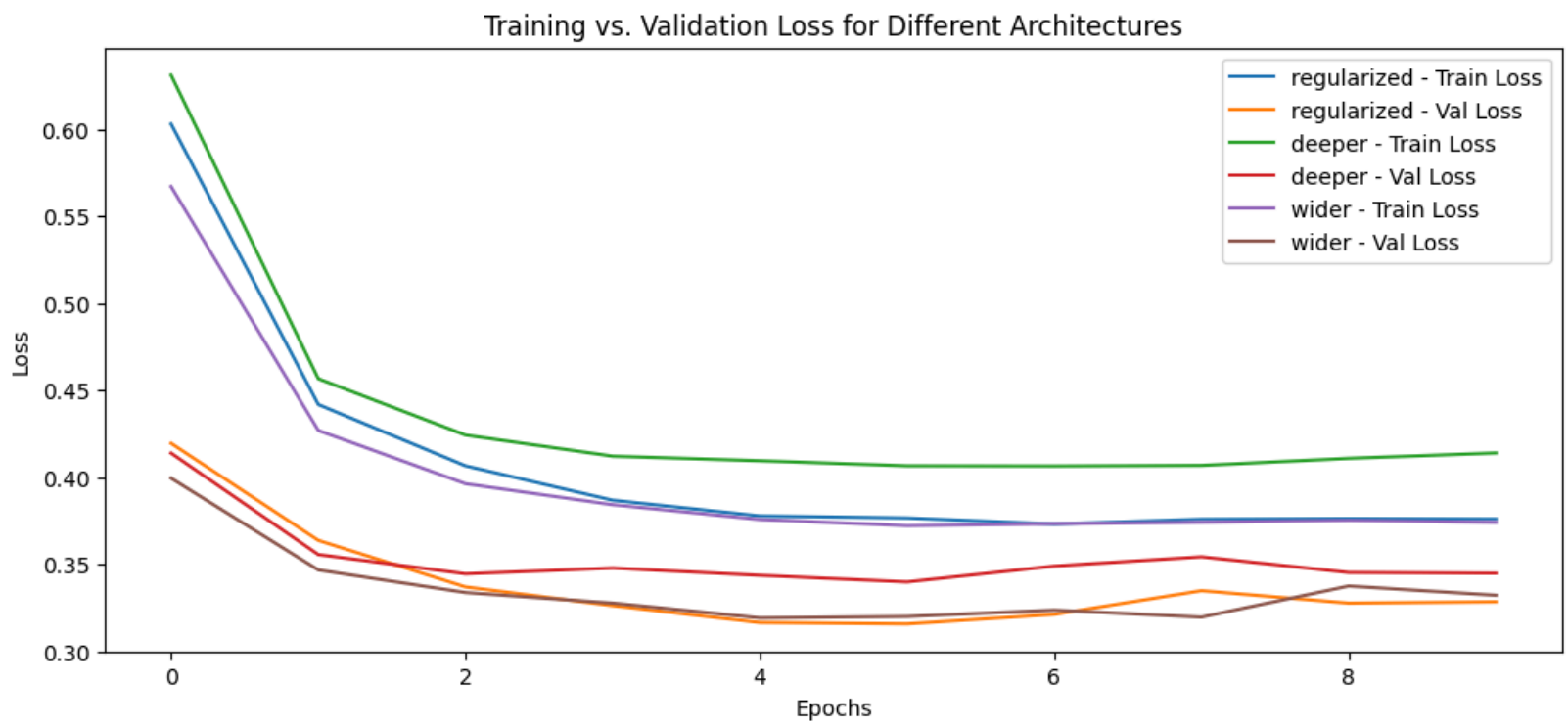
```python
for arch in architectures:
    print(f"\nTraining {arch} model...")
    model = build_model(arch, input_shape=(X_train_tfidf.shape[1],), output_units=y_train_one_hot.shape[1],
                        dropout=best_dropout, l2_reg=best_l2, learning_rate=best_learning_rate)

    history = model.fit(X_train_tfidf, y_train_one_hot,
                        validation_data=(X_val_tfidf, y_val_one_hot),
                        epochs=10, batch_size=best_batch_size, class_weight=class_weight_dict, verbose=1)

    # Make predictions
    y_pred = model.predict(X_test_tfidf)

    # Compute F1 Score
    f1 = f1_score(y_test_one_hot.argmax(axis=1), y_pred.argmax(axis=1), average='weighted')

    # Evaluate on test set
    test_loss, test_precision, test_recall = model.evaluate(X_test_tfidf, y_test_one_hot)

    # Store results
    results[arch] = {
        "test_loss": test_loss,
        "test_precision": test_precision,
        "test_recall": test_recall,
        "f1_score": f1,
        "history": history.history
    }

for arch, metrics in results.items():
    print(f"\n{arch.upper()} MODEL RESULTS:")
    print(f"Test Loss: {metrics['test_loss']:.4f}")
    print(f"Test Precision: {metrics['test_precision']:.4f}")
    print(f"Test Recall: {metrics['test_recall']:.4f}")
    print(f"F1 Score: {metrics['f1_score']:.4f}")
```

```
Training deeper model...
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Se
quential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 246s 19ms/step - Precision: 0.7873 - Recall: 0.5036 - loss: 0.7886 - val_Precision: 0.8946 - val_Recall: 0.8642 - val_loss: 0.4138
Epoch 2/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 264s 20ms/step - Precision: 0.8958 - Recall: 0.8611 - loss: 0.4641 - val_Precision: 0.9127 - val_Recall: 0.9009 - val_loss: 0.3555
Epoch 3/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 243s 19ms/step - Precision: 0.9111 - Recall: 0.8926 - loss: 0.4266 - val_Precision: 0.9165 - val_Recall: 0.9082 - val_loss: 0.3444
Epoch 4/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 262s 19ms/step - Precision: 0.9176 - Recall: 0.9025 - loss: 0.4109 - val_Precision: 0.9185 - val_Recall: 0.9098 - val_loss: 0.3478
Epoch 5/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 241s 19ms/step - Precision: 0.9212 - Recall: 0.9056 - loss: 0.4084 - val_Precision: 0.9210 - val_Recall: 0.9107 - val_loss: 0.3436
Epoch 6/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 268s 20ms/step - Precision: 0.9232 - Recall: 0.9080 - loss: 0.4054 - val_Precision: 0.9224 - val_Recall: 0.9139 - val_loss: 0.3398
Epoch 7/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 245s 20ms/step - Precision: 0.9244 - Recall: 0.9092 - loss: 0.4066 - val_Precision: 0.9249 - val_Recall: 0.9137 - val_loss: 0.3489
Epoch 8/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 244s 19ms/step - Precision: 0.9260 - Recall: 0.9096 - loss: 0.4043 - val_Precision: 0.9249 - val_Recall: 0.9140 - val_loss: 0.3542
Epoch 9/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 247s 20ms/step - Precision: 0.9249 - Recall: 0.9091 - loss: 0.4103 - val_Precision: 0.9257 - val_Recall: 0.9159 - val_loss: 0.3452
Epoch 10/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 253s 19ms/step - Precision: 0.9260 - Recall: 0.9095 - loss: 0.4076 - val_Precision: 0.9260 - val_Recall: 0.9138 - val_loss: 0.3448
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 9s 6ms/step - Precision: 0.9273 - Recall: 0.9144 - loss: 0.3401

Training wider model...
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Se
quential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 234s 19ms/step - Precision: 0.8218 - Recall: 0.5759 - loss: 0.7051 - val_Precision: 0.8948 - val_Recall: 0.8578 - val_loss: 0.3994
Epoch 2/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 240s 19ms/step - Precision: 0.8953 - Recall: 0.8591 - loss: 0.4360 - val_Precision: 0.9113 - val_Recall: 0.8862 - val_loss: 0.3467
Epoch 3/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 276s 20ms/step - Precision: 0.9109 - Recall: 0.8860 - loss: 0.3955 - val_Precision: 0.9193 - val_Recall: 0.8962 - val_loss: 0.3337
Epoch 4/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 259s 20ms/step - Precision: 0.9180 - Recall: 0.8952 - loss: 0.3860 - val_Precision: 0.9251 - val_Recall: 0.9027 - val_loss: 0.3276
Epoch 5/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 239s 19ms/step - Precision: 0.9244 - Recall: 0.9031 - loss: 0.3750 - val_Precision: 0.9285 - val_Recall: 0.9094 - val_loss: 0.3191
Epoch 6/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 261s 19ms/step - Precision: 0.9265 - Recall: 0.9072 - loss: 0.3705 - val_Precision: 0.9278 - val_Recall: 0.9140 - val_loss: 0.3199
Epoch 7/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 268s 19ms/step - Precision: 0.9293 - Recall: 0.9115 - loss: 0.3700 - val_Precision: 0.9286 - val_Recall: 0.9154 - val_loss: 0.3236
Epoch 8/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 244s 20ms/step - Precision: 0.9299 - Recall: 0.9122 - loss: 0.3723 - val_Precision: 0.9310 - val_Recall: 0.9159 - val_loss: 0.3195
Epoch 9/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 258s 19ms/step - Precision: 0.9311 - Recall: 0.9147 - loss: 0.3740 - val_Precision: 0.9307 - val_Recall: 0.9110 - val_loss: 0.3374
Epoch 10/10
12499/12499 ━━━━━━━━━━━━━━━━━━━━ 260s 19ms/step - Precision: 0.9310 - Recall: 0.9160 - loss: 0.3728 - val_Precision: 0.9297 - val_Recall: 0.9133 - val_loss: 0.3321
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 9s 6ms/step - Precision: 0.9317 - Recall: 0.9144 - loss: 0.3267

REGULARIZED MODEL RESULTS:
Test Loss: 0.3249
Test Precision: 0.9309
Test Recall: 0.9118
F1 Score: 0.9200

DEEPER MODEL RESULTS:
Test Loss: 0.3418
Test Precision: 0.9269
Test Recall: 0.9141
F1 Score: 0.9204

WIDER MODEL RESULTS:
Test Loss: 0.3281
Test Precision: 0.9315
Test Recall: 0.9146
F1 Score: 0.9209
```

In [44]:

```python
# Plotting the training and validation loss for the different architectures
plt.figure(figsize=(12, 5))
```

```
for arch in results.keys():
    plt.plot(results[arch]['history']['loss'], label=f'{arch} - Train Loss')
    plt.plot(results[arch]['history']['val_loss'], label=f'{arch} - Val Loss')

plt.title("Training vs. Validation Loss for Different Architectures")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()
```



Analysis of results

The graph compares the training and validation loss for three different model architectures: a regularized model, a deeper model, and a wider model. Across all architectures, the training loss decreases rapidly in the early epochs and then stabilizes, which is expected. The validation loss follows a similar trend but remains slightly higher, indicating some generalization gap. Among the three, the regularized model has the lowest validation loss throughout the training process, suggesting better generalization to unseen data. The deeper model, while initially showing a significant drop in loss, appears to plateau at a higher value compared to the other models, which might indicate diminishing returns from additional depth. The wider model, on the other hand, shows a trend similar to the regularized model but with slightly higher validation loss in later epochs.

From the test results, the regularized model achieves the lowest test loss (0.3249) and has strong performance across precision (0.9309), recall (0.9118), and F1 score (0.9200). The deeper model, despite having a slightly higher test loss (0.3418), achieves a comparable F1 score (0.9204), which suggests that increasing depth does not necessarily improve performance in this case. The wider model performs slightly better than the deeper model in terms of test loss (0.3281) and achieves the highest F1 score (0.9209), showing that increasing the width of the network can be beneficial but does not significantly outperform the regularized model.

Overall, the regularized model seems to be the best choice as it achieves the lowest test loss while maintaining strong precision and recall. The deeper model does not show significant improvement and might be overcomplicating the learning process, while the wider model performs well but does not provide a substantial advantage over the regularized model. This suggests that careful tuning of regularization techniques might be more effective than simply increasing depth or width.

References

Chollet, F. (2018). Deep learning with Python. Manning Publications.

Kaggle link to dataset: https://www.kaggle.com/datasets/khalidryder777/500k-chatgpt-tweets-jan-mar-2023/data